```
from google.colab import files
uploaded=files.upload()
```

Choose Files AI for QCM.xlsx
AI for QCM.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 8974 bytes, last modified: 3/26/2026 - 100% done
Saving AI for QCM.xlsx to AI for QCM.xlsx

```
import pandas as pd
#load excel file
data=pd.read_excel('AI for QCM.xlsx')
#check data
print(data.head())
print(data.columns)
print(data.info())
```

```
   Height (cm)  Weight (kg)
0          150           50
1          152           52
2          155           54
3          157           55
4          160           58
Index(['Height (cm)', 'Weight (kg)'], dtype='object')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 2 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Height (cm)  20 non-null     int64
 1   Weight (kg)  20 non-null     int64
dtypes: int64(2)
memory usage: 452.0 bytes
None
```

```
#input & output columns
x=data[['Height (cm)']]
y=data[['Weight (kg)']]
```

```
#Split the data
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
#view training data
print("x_train:\n",x_train)
print("y_train:\n",y_train)
```

```
x_train:
     Height (cm)
8           170
5           162
11          177
3           157
18          195
16          190
13          182
2           155
9           172
19          198
4           160
12          180
7           167
10          175
14          185
6           165
y_train:
     Weight (kg)
```

```
8            67
5            60
11           72
3            55
18           90
16           85
13           77
2            54
9            68
19           92
4            58
12           75
7            64
10           70
14           80
6            62
```

```
#view testing data
print("x_test:\n",x_test)
print("y_test:\n",y_test)
```

```
x_test:
     Height (cm)
0           150
17          192
15          187
1           152
y_test:
     Weight (kg)
0            50
17           87
15           82
1            52
```

```
#train regression model (linear regression)
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
#testing and evaluation
from sklearn.metrics import mean_squared_error,mean_absolute_error,r2_score
y_pred=model.predict(x_test)
print("MSE",mean_squared_error(y_test,y_pred))
print("r2 score",r2_score(y_test,y_pred))
```

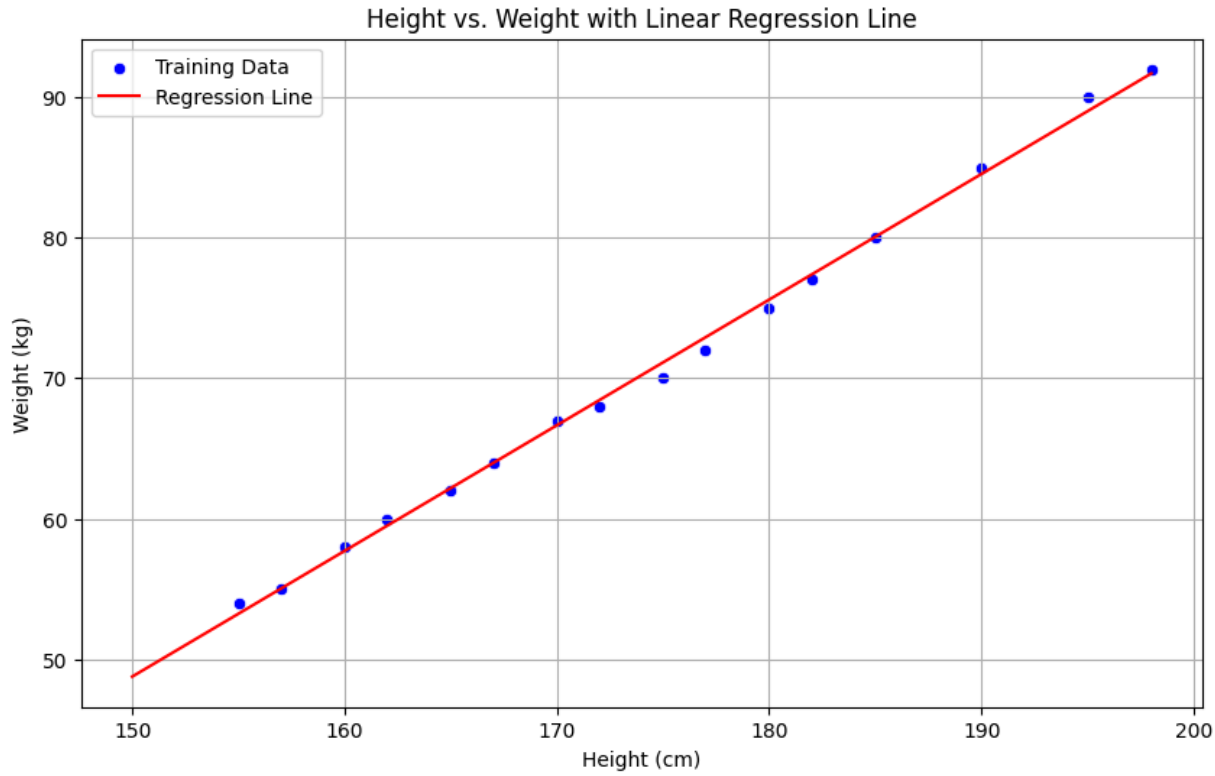
```
MSE 1.0355045277741066
r2 score 0.996356262932838
```

```
#predict new value(for x=250cm)
new_data=pd.DataFrame({'Height (cm)':[250]})
prediction=model.predict(new_data)
print("predicted value:",prediction)
```

```
predicted value: [[138.22565534]]
```

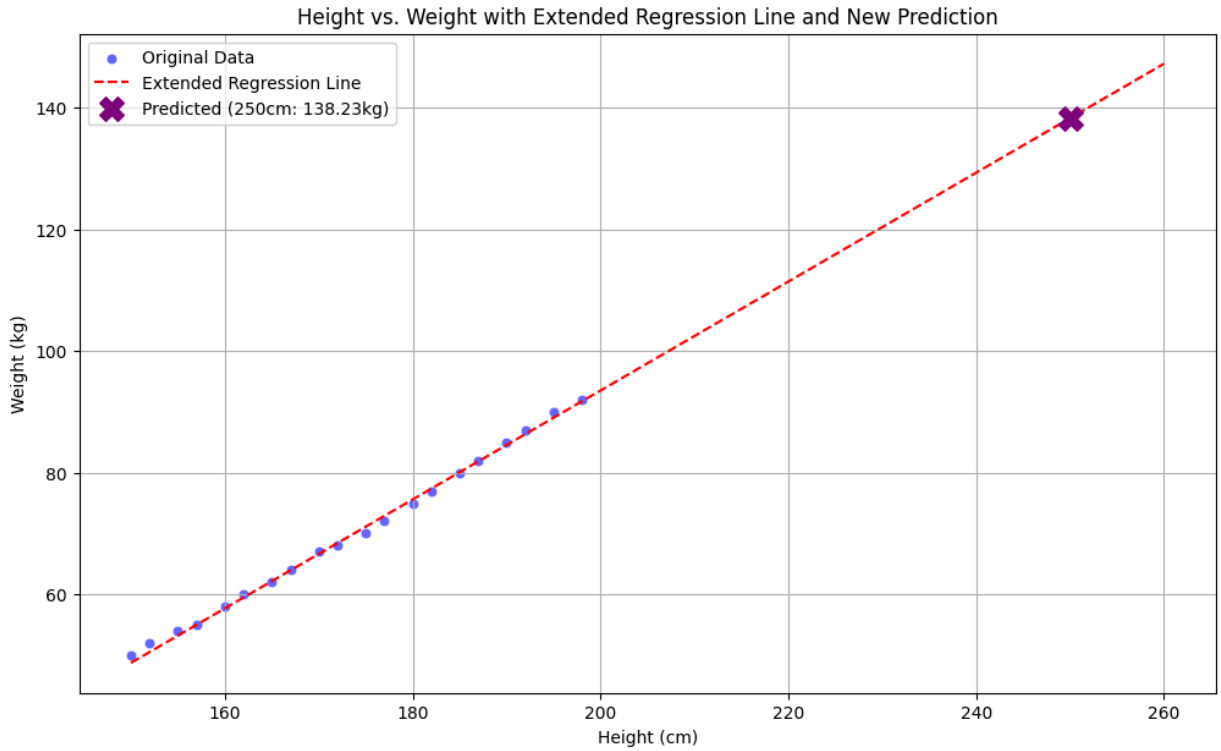
```
import matplotlib.pyplot as plt
import seaborn as sns

# Create a scatter plot of the training data
plt.figure(figsize=(10, 6))
sns.scatterplot(x=x_train['Height (cm)'], y=y_train['Weight (kg)'], label='Training Data', color='

# Plot the linear regression line
```

```
# Use the model to predict weights for a range of heights within the data's range
min_height = data['Height (cm)'].min()
max_height = data['Height (cm)'].max()
height_range = pd.DataFrame({'Height (cm)': [min_height, max_height]})
predicted_weights = model.predict(height_range)

sns.lineplot(x=height_range['Height (cm)'], y=predicted_weights.flatten(), label='Regression Line'

# Add labels and title
plt.xlabel('Height (cm)')
plt.ylabel('Weight (kg)')
plt.title('Height vs. Weight with Linear Regression Line')
plt.legend()
plt.grid(True)
plt.show()
```

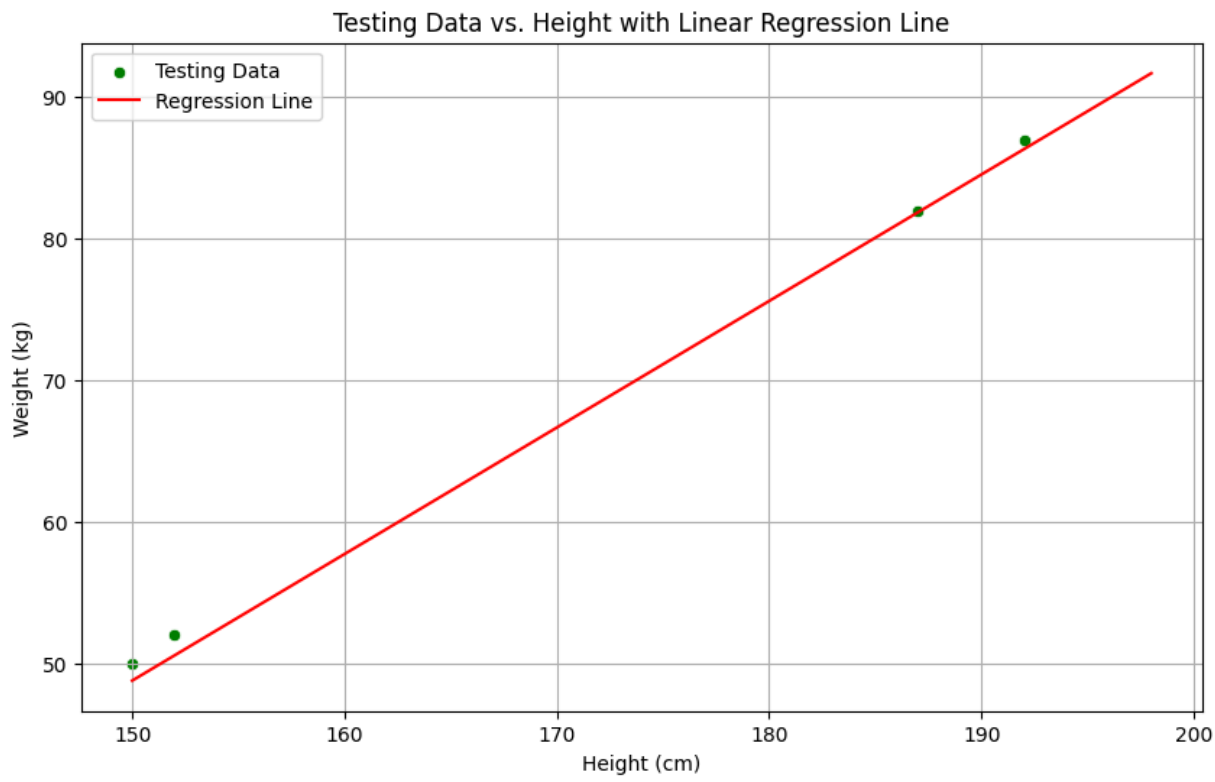



```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Create a scatter plot of all original data for context
plt.figure(figsize=(12, 7))
sns.scatterplot(x=data['Height (cm)'], y=data['Weight (kg)'], label='Original Data', color='blue',

# Extend the range for the regression line to include the prediction for 250cm
min_plot_height = data['Height (cm)'].min()
max_plot_height = max(data['Height (cm)'].max(), 250) + 10 # Ensure 250cm is well within the range

# Create a continuous range of heights for plotting the regression line
plot_height_range = pd.DataFrame({'Height (cm)': np.linspace(min_plot_height, max_plot_height, 100
predicted_weights_line = model.predict(plot_height_range)

sns.lineplot(x=plot_height_range['Height (cm)'], y=predicted_weights_line.flatten(), label='Extend

# Plot the specific prediction for 250cm
predicted_value_250 = model.predict(pd.DataFrame({'Height (cm)':[250]}))
plt.scatter(250, predicted_value_250, color='purple', marker='X', s=200, label=f'Predicted (250cm:

# Add labels and title
```

```
plt.xlabel('Height (cm)')
plt.ylabel('Weight (kg)')
plt.title('Height vs. Weight with Extended Regression Line and New Prediction')
plt.legend()
plt.grid(True)
plt.show()
```




```
import matplotlib.pyplot as plt
import seaborn as sns

# Create a scatter plot of the testing data
plt.figure(figsize=(10, 6))
sns.scatterplot(x=x_test['Height (cm)'], y=y_test['Weight (kg)'], label='Testing Data', color='gre

# Plot the linear regression line
# Use the model to predict weights for a range of heights within the data's range
min_height = data['Height (cm)'].min()
max_height = data['Height (cm)'].max()
height_range = pd.DataFrame({'Height (cm)': [min_height, max_height]})
predicted_weights = model.predict(height_range)

sns.lineplot(x=height_range['Height (cm)'], y=predicted_weights.flatten(), label='Regression Line'

# Add labels and title
plt.xlabel('Height (cm)')
plt.ylabel('Weight (kg)')
plt.title('Testing Data vs. Height with Linear Regression Line')
plt.legend()
plt.grid(True)
plt.show()
```


Testing Data vs. Height with Linear Regression Line
Testing Data
Regression Line
Weight (kg)
Height (cm)
90
80
70
60
50
150
160
170
180
190
200